Project 6.1 — CampusDesk-Student-Helpdesk-Ticket-Triage-Pipeline

A practice twin of Project 6 — LangChainPro. Same skills — the four LCEL patterns that turn single calls into a real pipeline: a **structured classifier** that routes each message, **parallel lookups** that run at once, **retries + a safe fallback** when a model call fails, and a **streamed reply** plus one **structured record** per message. A **new, real-world job**. **There are no steps here on purpose.** Read the story, build it yourself, and only open the hints or the solution if you get stuck.

Type: LCEL pipeline (classify → look up → reply → record) · **Stack:** Python, LangChain (LCEL: `with_structured_output`, `RunnableParallel`, `RunnableLambda`, `.with_fallbacks()`, `.stream()`), Pydantic · **Level:** Intermediate

> **Stuck, or want to check your work?** Full hints · Complete solution

The Story — what you are going to build

A university help desk gets hundreds of student messages a day: *"How do I add a class?"*, *"My aid disbursement is late"*, *"I'm locked out of the portal"*. A person reads each one, figures out **which office it belongs to**, pulls up the student's **record, open requests, and next deadline**, and writes a reply. When the system they rely on is **down**, they don't go silent — they send a holding note and flag it for a human. And anything **urgent or unclear** gets escalated, not auto-answered.

You are going to build **CampusDesk**: that triage desk as an LCEL pipeline. Each message flows through four stages, each one a LangChain pattern:

- **Classify** — a structured classifier sorts the message into `enrollment`, `financial_aid`, or `it_support`, with a reason and a `sure` flag, and routes it to the matching reply chain.
- **Look up** — three lookups (the student's status, their open requests, the next key deadline) run **at the same time**, not one after another, so the reply has full context fast.
- **Reply** — the right per-category chain drafts a reply. If the model call fails, a **retry then a safe canned reply** takes over instead of crashing or going silent.
- **Record** — the reply **streams** to the screen live, and the whole thing is captured as one **structured record** per message, with flags for "used the fallback" and "needs a human" (set when the classifier wasn't sure, or the message looks urgent).

The one lesson of this project: a real pipeline isn't one model call — it's *routing*, *doing independent work in parallel*, *degrading safely when a call fails*, and *leaving a structured trail*. LCEL

gives you each of those as a pattern instead of code you hand-write and debug for every chain.

How it works, from a message to a record (picture the whole machine)

Read this once and the whole project should form in your head.

A student message comes in. The **classifier chain** — `prompt | model.with_structured_output(TicketCategory)` — returns a fixed shape: a category (one of three, enforced by a `Literal`), a reason, and `sure`. Because the model must answer in that shape, you never parse free text, and a wrong value is caught, not stored. The classifier is wrapped with a **fallback**: if the structured parse fails, a `RunnableLambda` returns a safe "unsure" category so the pipeline keeps moving and the message gets flagged for a human.

With the category known, the **lookups** run. Instead of fetching the student's status, then their open requests, then the next deadline one after another (three waits), a `RunnableParallel` fires all three **at once** and hands back one merged dict — roughly the time of the slowest, not the sum.

The category picks the **reply chain** (`prompt | model | StrOutputParser()`), drafted with the message and the looked-up context. That chain is wrapped with `.with_fallbacks()`: the model call is retried, and if it still fails, a **canned holding reply** is returned and the record is marked `used_fallback`. The reply **streams** token by token to the screen (`.stream()` with `flush=True`), so the user sees it appear live.

Finally, everything becomes one **structured record**: student, message, category, reply, `used_fallback`, and `needs_human_review` with reasons — set true when the classifier wasn't sure, or the message contains an urgent signal ("deadline today", "locked out", "dropped"). That record is what a human or a dashboard reads later.

That's the whole machine: **classify into a fixed shape (with an unsure fallback) → run the lookups in parallel → draft the reply (retry, then canned fallback) → stream it → capture one structured record with the right flags.**

What you are building (the pieces)

- A **structured classifier** (`with_structured_output(TicketCategory)`) that routes to one of three reply chains, wrapped with an **unsure fallback**.
- **Parallel lookups** (`RunnableParallel` of three `RunnableLambda`s) that merge into one context dict.
- **Per-category reply chains** (`prompt | model | parser`), each wrapped with `.with_fallbacks()` → a retry and a safe canned reply.

- **Streaming** of the reply (`.stream()`), and one **structured** `TicketRecord` per message with `used_fallback` / `needs_human_review` / `review_reasons` .

Inputs: a student message (and a small local students file). **Outputs:** a streamed reply and a structured record per message. **Rules:** the category is a fixed shape; lookups run in parallel; a failed model call degrades to a canned reply, never a crash; unsure or urgent messages are flagged for a human.

A real example to build and test against

- *"How do I add a second class this term?"* → `enrollment` , routed to the enrollment reply chain; a normal streamed reply; record not flagged.
- *"My financial aid hasn't arrived and the tuition deadline is today."* → `financial_aid` , **and** `needs_human_review` (urgent signal "deadline today").
- A message the classifier reads half as billing, half as IT → classifier returns `sure=False` → flagged for a human, not quietly auto-answered.
- The model deliberately made to fail → the reply chain retries, returns the canned holding reply, and the record is marked `used_fallback` .

How to practice this

1. Read this story. **Don't open the solution yet.**
2. Plan it on paper: the four stages, the fixed category shape, which work can run in parallel, and exactly when a message gets flagged for a human.
3. Build it yourself. A good order: the classifier + routing → parallel lookups → retry + fallback → streaming + the structured record. Your call.
4. Test it on the real example: a clean route, an urgent flag, an unsure flag, and a forced model failure using the fallback.
5. **Only if stuck**, read the hints. **To compare**, read the complete solution.

Why this project is useful (read this at the end)

Almost every real LLM feature is a *pipeline*, not a single call: it routes the input, gathers context (often from several places at once), handles the call failing without going dark, and leaves a structured trail for humans and dashboards. CampusDesk is that shape — a help desk here, but the same four patterns power support triage, claims intake, order routing, and inbox assistants everywhere. Doing it in LCEL means parallelism, fallbacks, and streaming are each one line you can trust, not bespoke code you debug per chain. And the "flag unsure or urgent for a human" habit is what keeps an auto-responder from confidently mishandling the one message that actually mattered.

Done when

- A message is classified into the fixed shape and routed to the matching reply chain; the record captures it.
- The three lookups run in parallel (clearly faster than one-after-another), merged into one context.
- A forced model failure retries and falls back to the canned reply, with `used_fallback` set — no crash.
- An unsure classification and an urgent message are both flagged `needs_human_review` with reasons.
- Your tests pass: settings, the schemas, the parallel lookups (merge + timing), the fallbacks, the review-flag logic, and a live run.

The full build, every file, is in the complete solution — use it to check your work, not to skip the thinking.